

Tecnología Digital VI

Introducción a Redes Neuronales

Licenciatura en Tecnología Digital - UTDT

Del mundo tabular al no-estructurado

En la mayoría de los problemas vistos hasta ahora, las columnas del dataset representaban diferentes datos (edad, altura, nacionalidad) con una semántica muy ad-hoc que dificultaba una generalización *a priori* de la interacción entre los features. En el mundo tabular, dejábamos que los modelos (generalmente basados en árboles), vayan buscando las interacciones iterativamente dentro de cada subregión.

Cuando trabajamos con datos “no-estructurados” como imágenes, texto, audio, sabemos que hay una relación general más allá del problema puntual que esté tratando (localización espacial, localización temporal, etc).

Redes Neuronales

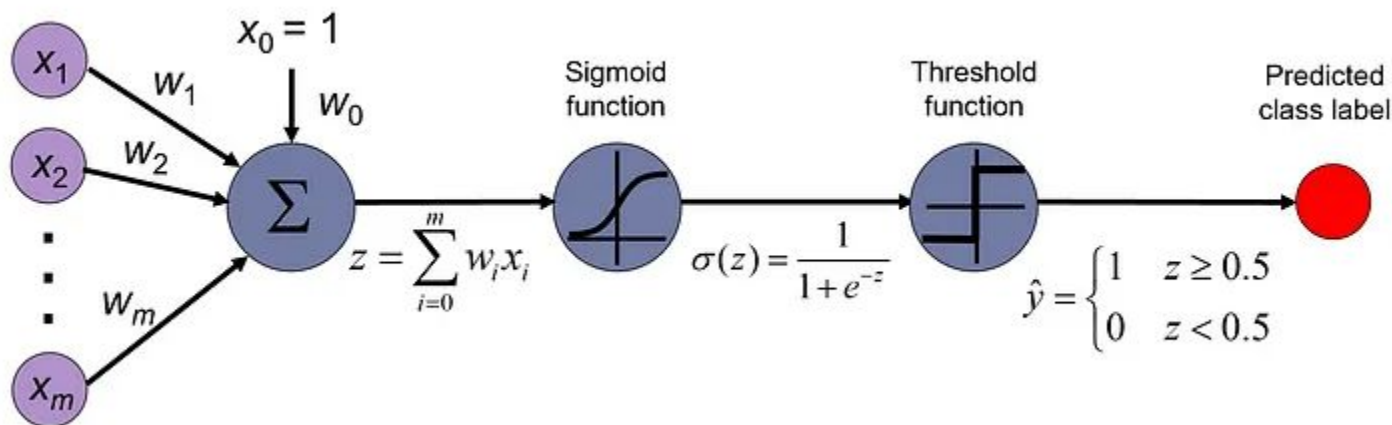
En esta segunda parte de la materia vamos a continuar nuestro camino de modelos conociendo las Redes Neuronales, que en la actualidad son EL modelo estrella dentro la Inteligencia Artificial. Si bien las podemos aplicar sobre cualquier problema de los vistos hasta ahora (regresión, clasificación, clusterización, etc), van a resultar particularmente performantes en problemas de datos no-estructurados.

Para comenzar a entender el formalismo, y también por una cuestión histórica, vamos a arrancar con un problema de clasificación binaria. A partir de ahí veremos cómo se puede extender a los demás tipos de problemas.

Regresión Logística

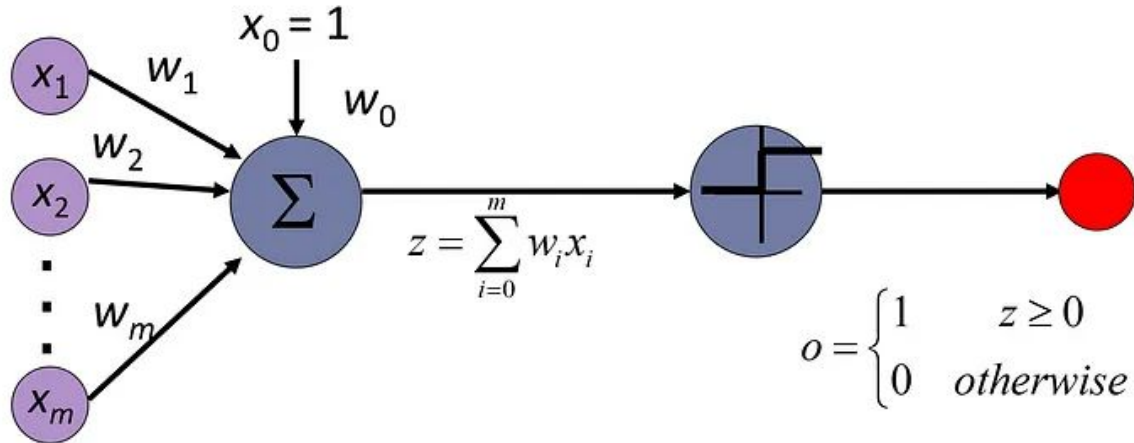
Nuestro camino de clasificación binaria comenzó con la Regresión Logística.

Recordemos la arquitectura de ese modelo. El objetivo era, dado un dataset, encontrar los mejores w_i usando la NLL como *loss function*. Luego, todo lo demás ya está fijado.



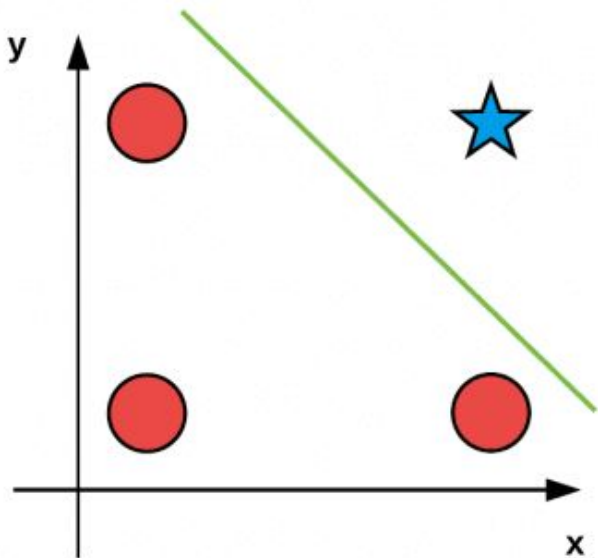
Perceptrón

Veamos ahora **la arquitectura** de nuestra primera red neuronal (formalmente una sola neurona). Es aún **más simple en sus cuentas que la Regresión Logística**. Consta de una combinación lineal de los features más un bias, y luego la aplicación de una *Heaviside step function*.

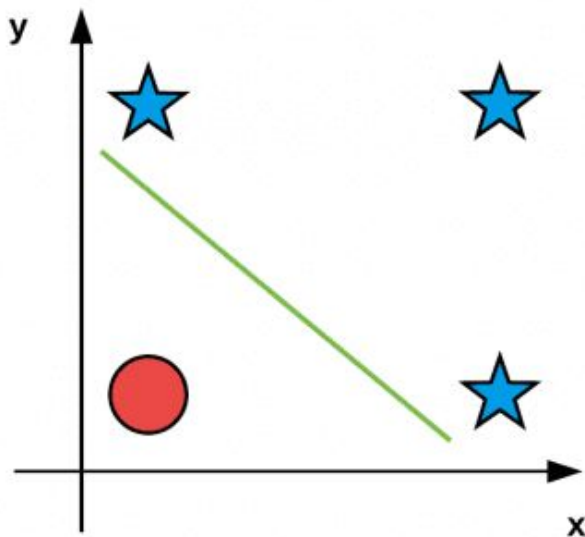


Datasets lógicos

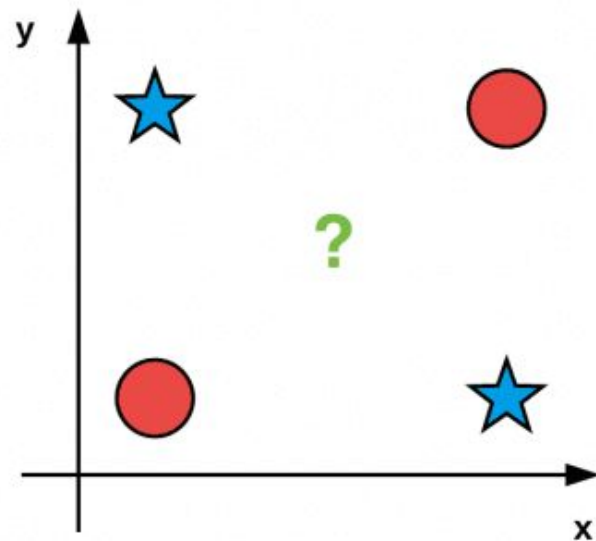
AND



OR



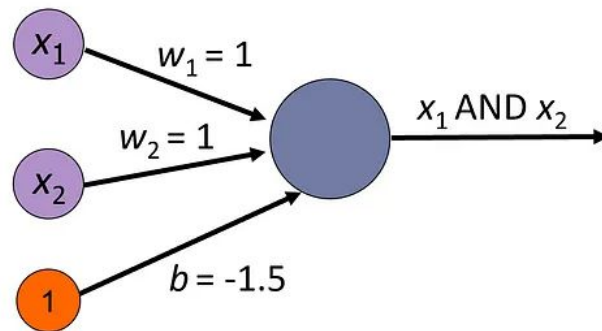
XOR



Perceptrón “And”

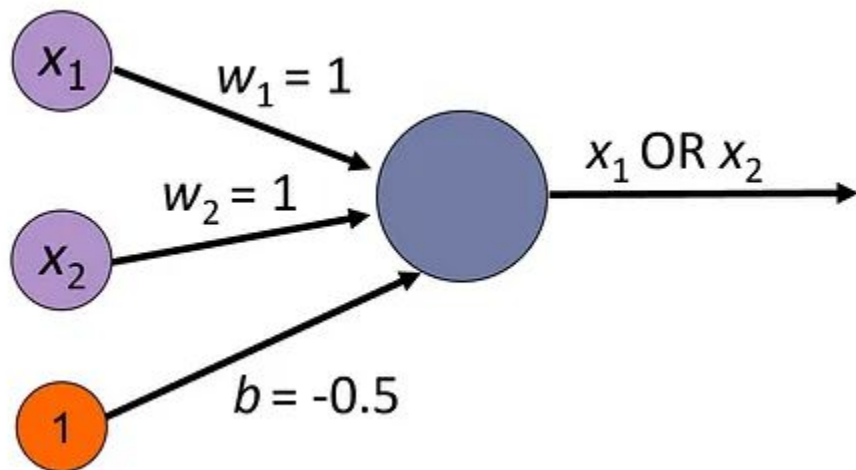
Si bien no sabemos cómo se entrena un perceptrón, por el momento podemos confiar en que existe una manera. Adicionalmente para un problema simple como el “and” lógico, podemos ver *a ojo* cómo es un perceptrón entrenado.

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1



Perceptrón “Or”

El caso del “or” lógico es parecido. Es simple ver cómo sería el perceptrón entrenado.



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

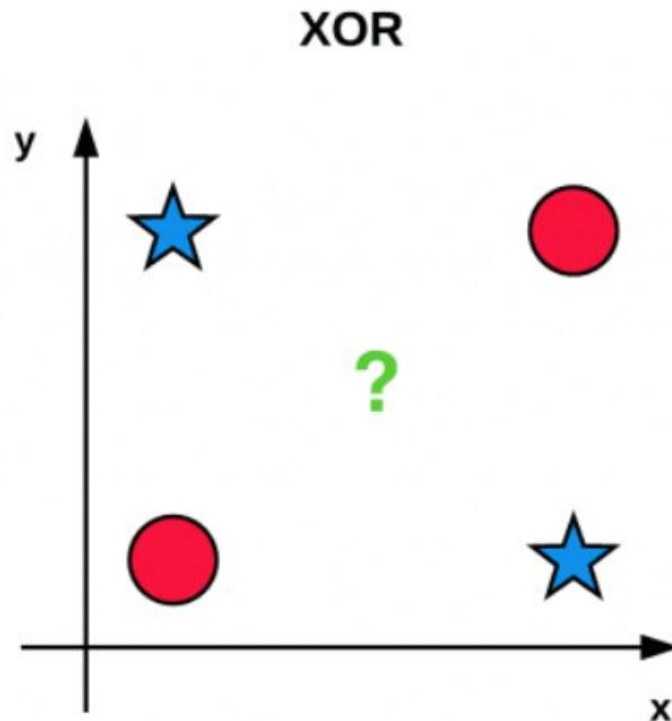
Perceptrón't "xor"

Para el "xor" lógico no encontramos un perceptrón entrenado que ajuste al dataset.

Es fácil demostrar que este perceptrón no puede existir, considerando:

- $W_1 + b \geq 0$
- $W_2 + b \geq 0$
- $b < 0$

Entonces, ...



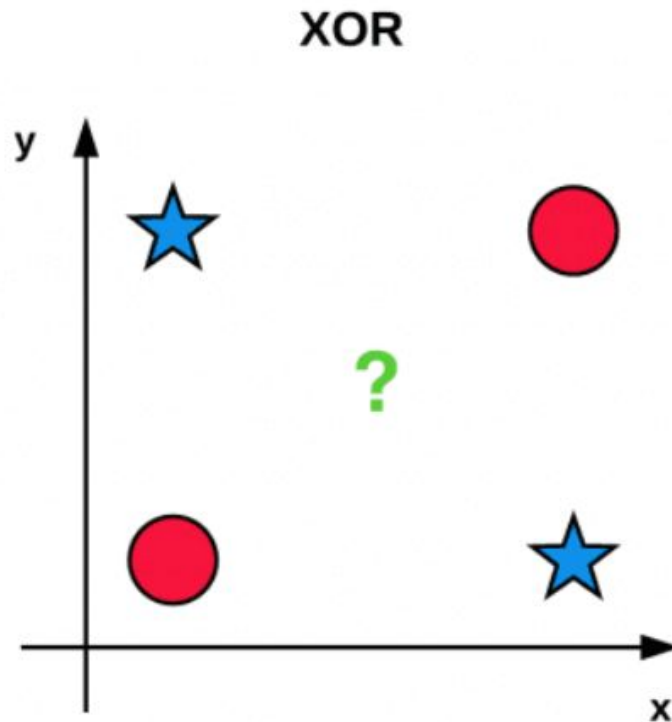
Perceptrón't “xor”

Para el “xor” lógico no encontramos un perceptrón entrenado que ajuste al dataset.

Es fácil demostrar que este perceptrón no puede existir, considerando:

- $W_1 + b \geq 0$
- $W_2 + b \geq 0$
- $b < 0$

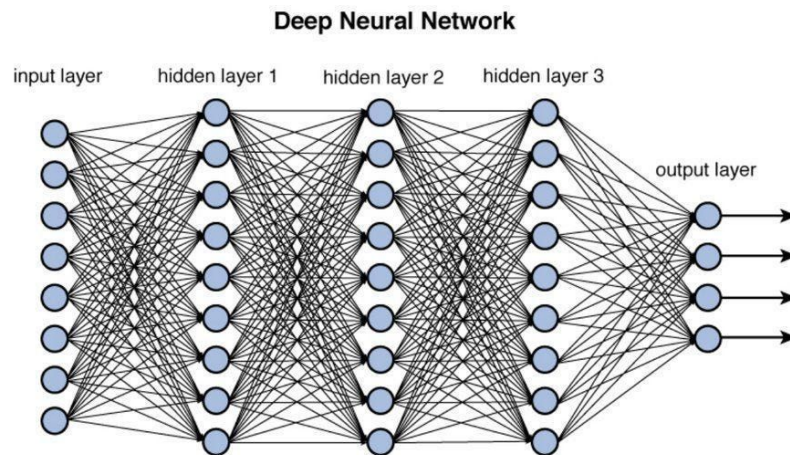
Entonces, $W_1 + W_2 + b \geq W_1 + b + W_2 + b \geq W_2 + b \geq 0$.



Arquitecturas

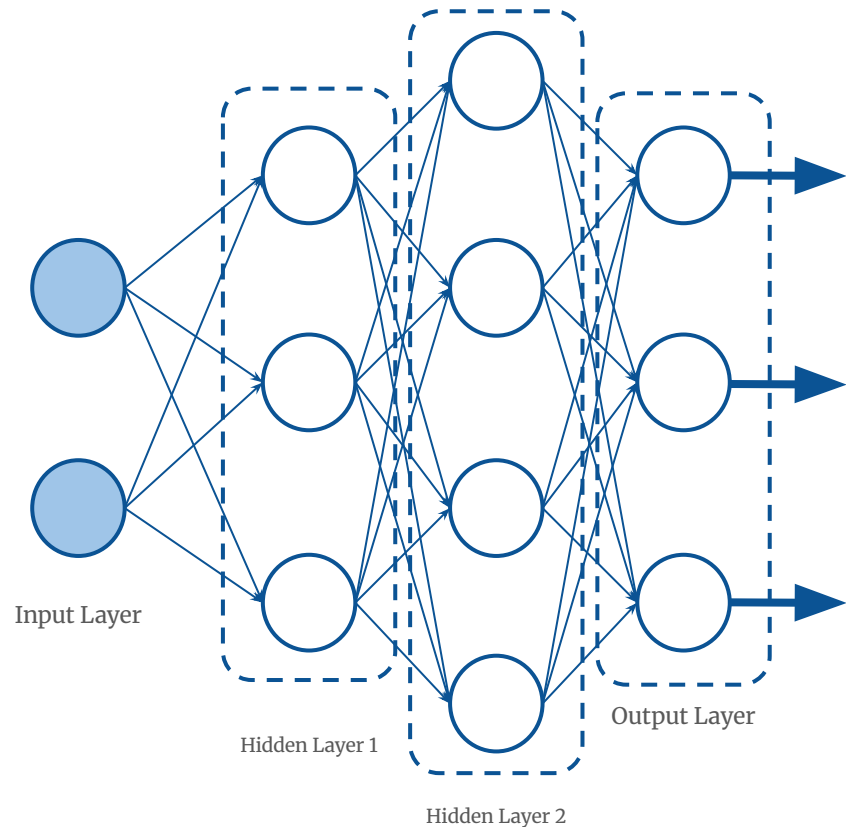
Una neurona sola no puede hacer demasiado. ¿Y si ponemos neuronas en paralelo? ¿y si ponemos neuronas en serie? ¿y si armamos redes de varias neuronas conectadas?

Podemos interconectar varios nodos en donde cada uno sea una neurona (combinación de inputs + función de activación).



Perceptrón Multicapa

- Para juntar neuronas en paralelo, vamos a utilizar el concepto de **capa (layer)**. Es la existencia de **varias capas** (particularmente las **ocultas**) las que definen el paso al **Deep Learning** (o *Aprendizaje Profundo*).
- **Capaz fully-connected** (FC).
- **Red Feed-Forward**.



Funciones de activación

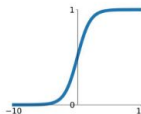
No solo podemos modificar la arquitectura sino también la función de activación.

Las funciones de activación son muy importantes en la arquitectura porque son las que introducen la no linealidad (combinaciones de combinaciones lineales, siguen siendo lineales).

Activation Functions

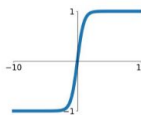
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



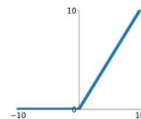
tanh

$$\tanh(x)$$



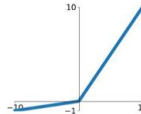
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

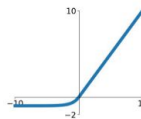


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Backpropagation

En un determinado momento del área (circa 1970) surgían arquitecturas conexionistas más complejas que el perceptrón pero no quedaba claro cómo entrenarlas (o era muy compleja la manera).

El algoritmo de Backpropagation es la pieza fundamental de las redes neuronales que nos permite saber cómo actualizar los diferentes pesos de la red en base a ir propagando los diferentes gradientes por la misma.

De esta manera podemos empezar a utilizar métodos como Gradient Descent (y los derivados que mencionamos) para optimizar diferentes loss functions.

Baseline Regresión Logística

Como mencionamos al comenzar, podemos **empezar por un problema de clasificación binaria.**

Usamos el dataset breast cancer presente en Scikit-learn y utilizamos el modelo de Regresión Logística.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Load breast cancer dataset from scikit-learn
data = load_breast_cancer()
X = data.data
y = data.target

# Perform feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split the dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# Create an instance of the logistic regression model
model = LogisticRegression()

# Train the model
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Calculate the accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")

Accuracy: 0.9736842105263158
```

Arquitectura

Lo siguiente es definir la arquitectura de la red neuronal. En este caso estamos usando el módulo Sequential de PyTorch que permite ir pegando capas de manera secuencial.

Primero ponemos una capa lineal a la que entren tantos valores como tenga el input y solo salga un único valor.

Luego sumamos una capa sigmoidea.

```
# Define the sequential model
model = nn.Sequential(
    nn.Linear(X_train.shape[1], 1),
    nn.Sigmoid()
)
```


Entrenamiento

Definimos la cantidad de épocas que vamos a entrenar.

Por cada época:

- Inferimos y calculamos la loss.
- Limpiamos todos los gradientes que traíamos.
- Hacemos el backpropagation de los gradientes según la loss.
- Hacemos un paso de SGD.

```
# Train the model
num_epochs = 1000
for epoch in range(num_epochs):
    # Forward pass
    outputs = model(X_train)
    loss = criterion(outputs, y_train)

    # Backward and optimize
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 100 == 0:
        print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item()}")
```

Evaluación

En primer lugar, con `torch.no_grad()` le avisamos a la librería que ya estamos en tiempo de inferencia y que no queremos modificar ningún parámetro.

Luego hacemos las predicciones y medimos nuestro resultado final.

```
# Evaluate the model on the test set
with torch.no_grad():
    outputs = model(X_test)
    predicted = (outputs >= 0.5).float()
    accuracy = accuracy_score(y_test, predicted)
    print(f"Accuracy: {accuracy}")
```

```
Epoch [100/1000], Loss: 0.24800749123096466
Epoch [200/1000], Loss: 0.19175125658512115
Epoch [300/1000], Loss: 0.16453160345554352
Epoch [400/1000], Loss: 0.14790794253349304
Epoch [500/1000], Loss: 0.1364613026380539
Epoch [600/1000], Loss: 0.12798260152339935
Epoch [700/1000], Loss: 0.12138690054416656
Epoch [800/1000], Loss: 0.11607246100902557
Epoch [900/1000], Loss: 0.11167559027671814
Epoch [1000/1000], Loss: 0.10796193778514862
Accuracy: 0.9912280701754386
```